

VendorFlow

Project Proposal

A full-stack SaaS platform that helps local grocery stores and pharmacies digitize their business operations — inventory, online orders, delivery coordination, and sales analytics.

Prepared by

Mannat

Document

Course Project Proposal

Date

September 2026

4

user portals

3

ER diagram levels

4

language stack

9 wks

build timeline

1 Project Overview

2 Scope & Feature Set

Version 1 (MVP) and Version 2 (future)

3 Technology Stack

C++, Java, Python, TypeScript

4 Data Model

Figures 1–3: Conceptual, Logical & Physical ER Diagrams

5 Working Prototype

Figures 4–7: Login, Customer, Vendor & Admin portals

6 Project Timeline

Figure 8: Development Gantt Chart

1. Project Overview

Instead of every local grocery store or pharmacy building its own website or app, businesses join VendorFlow to manage inventory, receive online orders, coordinate deliveries, and analyze sales through a single shared platform. Customers can discover nearby stores, upload prescriptions for pharmacy products, place orders, and track deliveries end to end.

The platform serves four kinds of users — **Customers**, **Vendors**, **Delivery Partners**, and **Admins** — each with a dedicated portal, and is backed by a role-based access control system so every user only sees what's relevant to them.

2. Scope & Feature Set

Version 1 — Production MVP

- Customer, Vendor, Delivery Partner & Admin portals
- JWT authentication & role-based access control
- Grocery & pharmacy support
- Vendor onboarding & approval workflow
- Product & inventory management
- Smart vendor routing (distance + stock + reliability)
- Vendor reliability score
- Prescription upload & manual verification
- Shopping cart & checkout
- Complete order lifecycle management
- Delivery assignment & status-based tracking
- Real-time notifications (Socket.IO)
- Email & in-app notifications
- Reviews & ratings
- Vendor & admin analytics dashboards
- Redis caching & rate limiting
- Background jobs (notifications, routing timeout, reliability updates)
- Audit logs & activity history
- Production deployment

Version 2 — Future Enhancements

- Support for bakery, hardware, electronics & other local businesses
- Payment gateway integration
- AI sales & inventory assistant
- Demand forecasting
- Personalized product recommendations
- Mobile / PWA app for delivery partners
- Bulk product import (CSV / Excel)
- Multi-language support
- Advanced business analytics & reports
- Customer loyalty & rewards program
- Coupons & promotional campaigns
- Performance optimization & load scaling

3. Technology Stack

Layer	Technology	Purpose
Core backend	C++, Java	High-performance API and business logic layer
Routing & jobs	Python	Smart vendor routing, reliability scoring, background job workers
Frontend	TypeScript	Customer, vendor, delivery partner & admin portals
Primary database	PostgreSQL	Relational storage for users, orders, inventory, wallets
Cache & queues	Redis	Caching, rate limiting, background job queues
Real-time layer	Socket.IO	Live order status & notification updates

Why this split

The stack is split by workload rather than by convenience: each language is doing the job it's best suited for, and the pieces talk to each other over well-defined boundaries.

- **C++ and Java** together handle the core API, authentication, and order/inventory business logic — C++ for the performance-critical paths, Java for the broader service layer and easier integration with enterprise tooling.
- **Python** runs as a separate service for smart vendor routing, reliability scoring, and the background job workers — tasks that change often and benefit from a faster iteration cycle.
- **TypeScript** powers all four frontend portals (customer, vendor, delivery partner, admin), sharing a single type-safe API contract with the backend.
- **PostgreSQL** is the single source of truth for all relational data; **Redis** sits in front of it for caching, rate limiting, and queuing background jobs between the C++ and Python services.
- **Socket.IO** pushes order-status and notification events to connected clients in real time — so a request flows from the TypeScript frontend to the C++ API, with vendor-matching delegated to the Python service and Redis/PostgreSQL as shared state underneath.

4. Data Model

The schema is presented at three levels of detail, moving from a high-level conceptual map down to the exact columns and data types used in the database:

- **Conceptual (Figure 1)** — just the entities and how they relate, useful for explaining the system to a non-technical audience.
- **Logical (Figure 2)** — adds primary/foreign keys and the junction tables needed to actually implement the relationships (cart items, order items, delivery assignments).
- **Physical (Figure 3)** — the full column list with data types, ready to translate directly into PostgreSQL CREATE TABLE statements.

Entity	Purpose
User	Base account for every person on the platform, with a role (customer / vendor / delivery / admin)
Vendor	A store's profile — type, location, approval status, reliability score
Product / Inventory	Items a vendor sells and the stock level behind each one
Cart / Cart Item	A customer's in-progress selection before checkout
Order / Order Item	A placed order and the line items within it
Prescription	Uploaded pharmacy prescription, pending or verified by an admin
Delivery Assignment	Links an order to the delivery partner fulfilling it, with live status
Wallet / Transaction	A user's running balance and the credit/debit history behind it
Review	A customer's rating and comment tied to a completed order

Figure 1 — Conceptual ER Diagram (Level 1)

Entities and how they relate — no keys or attributes, the high-level map of the system.

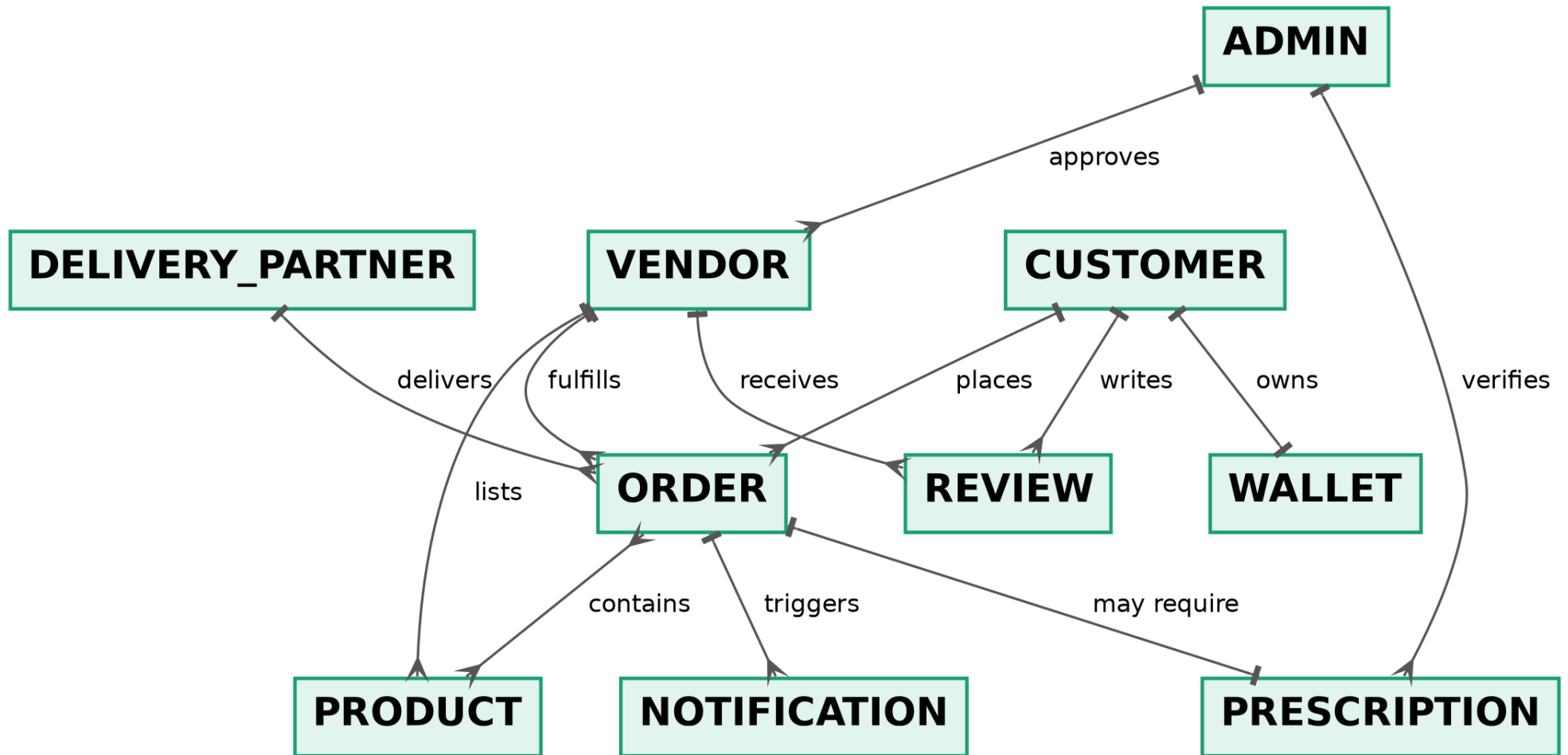


Figure 2 — Logical ER Diagram (Level 2)

Adds primary/foreign keys and the junction tables (cart, order items, delivery assignment) needed to implement the relationships.

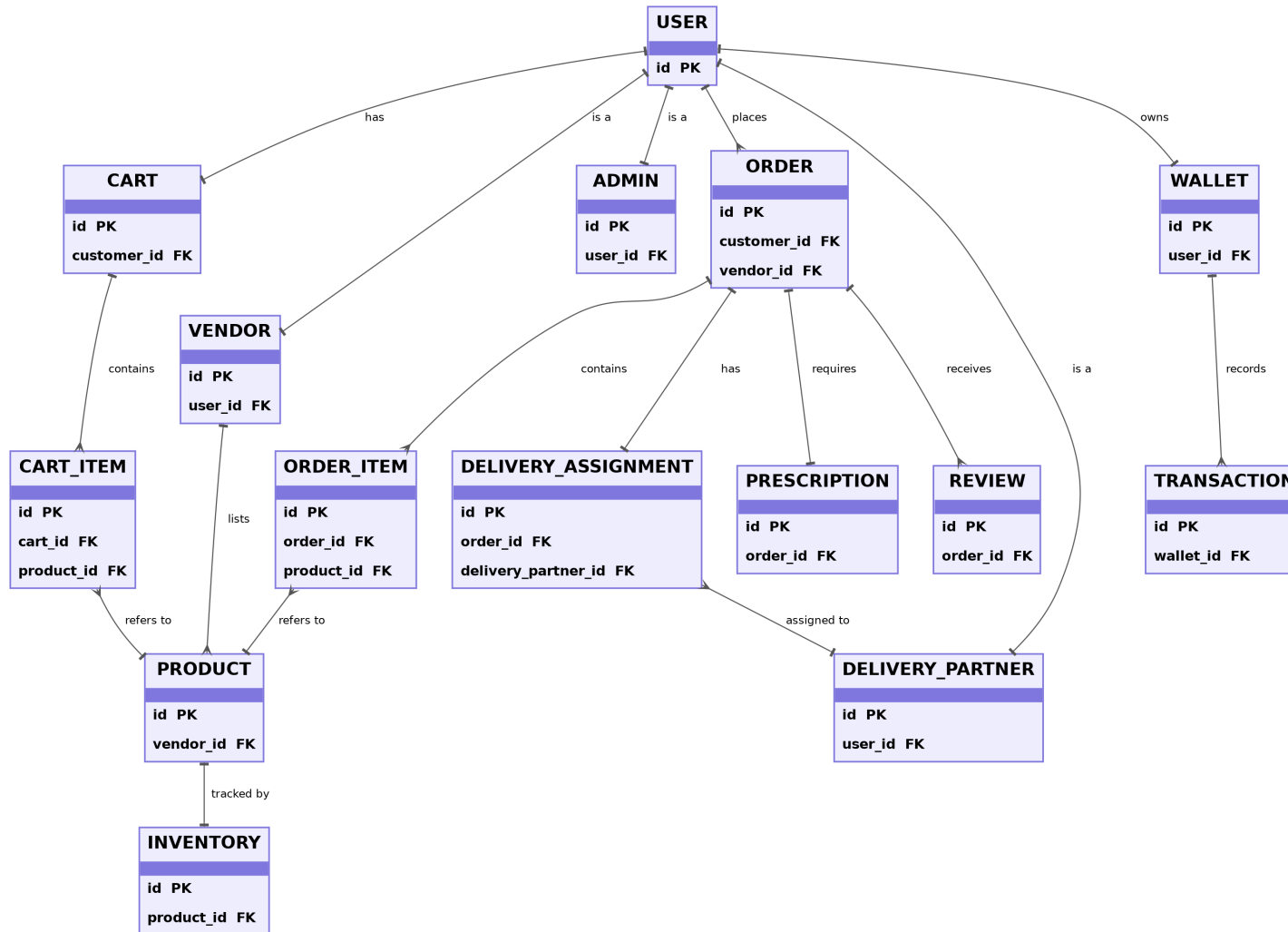
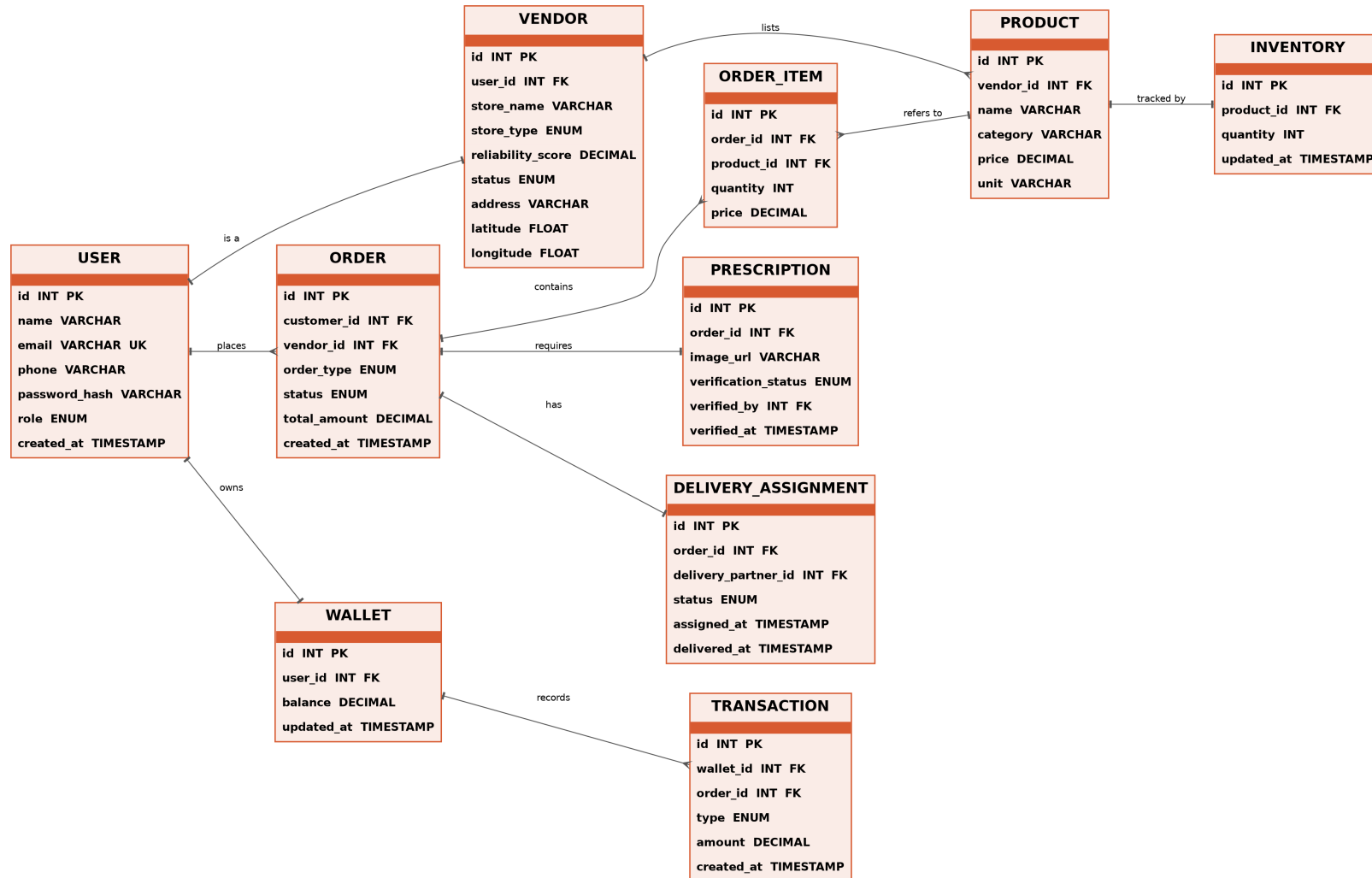


Figure 3 — Physical ER Diagram (Level 3)

Full column definitions with data types — the schema ready to translate into CREATE TABLE statements.



5. Working Prototype

Screens from the current build of VendorFlow, covering authentication, the customer shopping flow, the vendor inventory panel, and the admin approval dashboard. Each screen maps directly back to a use case from the platform's core feature set:

Figure	Screen	Demonstrates
4	Login & authentication	Role-based sign-in and JWT-backed session handling
5	Customer — Browse Products	Product discovery across multiple approved vendors, with cart
6	Vendor — Dashboard	Store profile, approval status, and product/inventory management
7	Admin — Dashboard	Vendor approval queue and reliability-score oversight

Delivery-partner and analytics-dashboard screens are implemented behind the same authentication flow and follow in the next iteration of this write-up once their UI is finalized.

Figure 4 — Login & Authentication

Role-based sign-in — the same login screen routes customers, vendors, delivery partners, and admins to their respective portals.

VendorFlow

LoginRegister

Welcome back

Sign in to your VendorFlow account

Email

you@example.com

Password

Sign In

Don't have an account? [Register](#)

Demo Credentials (password: password123)

-  admin@vendorflow.com — Admin
-  quickmart@vendorflow.com — Vendor
-  freshstore@vendorflow.com — Vendor
-  customer@vendorflow.com — Customer

Figure 5 — Customer Portal — Browse Products

Customers can search across approved local stores and add items from grocery and pharmacy vendors to a single cart.

VendorFlow

Browse  **Test Customer** **customer** Logout

Browse Products

From approved local stores

Search products or categories...

Apple ₹50

Fruits

 QuickMart

 In Stock (2)

Add to Cart

Banana ₹20

Fruits

 QuickMart

 In Stock (30)

Add to Cart

Milk ₹55

Dairy

 QuickMart

 In Stock (10)

Add to Cart

Apple ₹48

Fruits

 FreshStore

 In Stock (50)

Add to Cart

Banana ₹18

Fruits

 FreshStore

 In Stock (100)

Add to Cart

Milk ₹52

Dairy

 FreshStore

 In Stock (60)

Add to Cart

Bread ₹35

Bakery

 FreshStore

 In Stock (40)

Add to Cart

Figure 6 — Vendor Portal — Dashboard

Vendors manage their store profile, approval status, and product catalog, including adding new products and tracking stock.

VendorFlow

DashboardQuickMart OwnervendorLogout

Vendor Dashboard

QuickMart

Grocery · 28.6139, 77.2090

APPROVED

+ Add Product

Product Name

Apple

Category

Fruits

Price (₹)

50.00

Stock Quantity

100

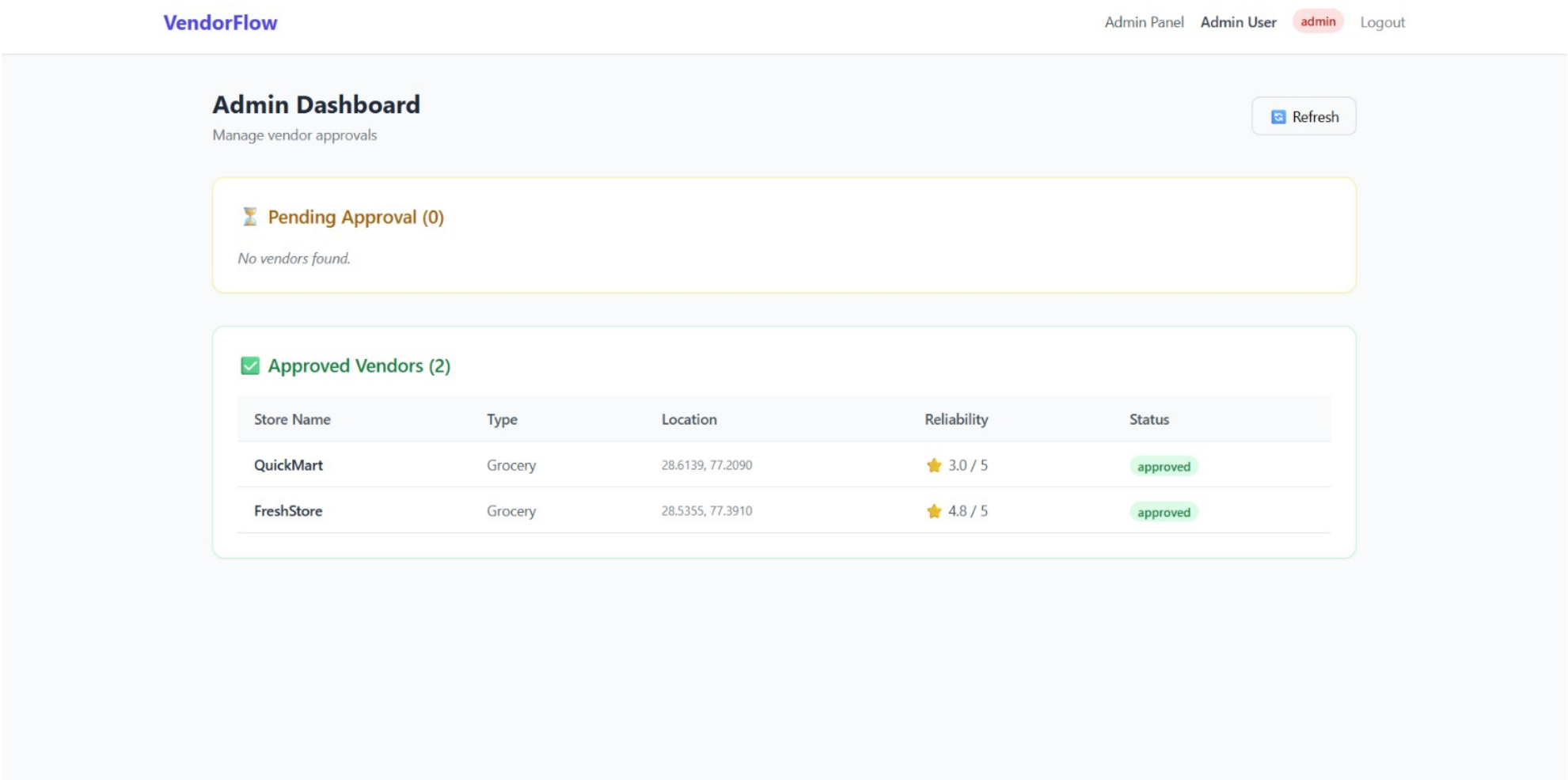
Add Product

📦 My Products (3)

Name	Category	Price	Stock	Actions
Apple	Fruits	₹50	2	Edit

Figure 7 — Admin Portal — Dashboard

Admins review pending vendor applications and monitor approved vendors, including their reliability scores.



6. Project Timeline

Development is scoped to roughly nine weeks, split across the languages in use: C++ and Java for the core backend, Python for routing and background jobs, and TypeScript for the frontend portals. Work on the backend and frontend tracks runs in parallel once the shared data model is finalized.

Phase	Track	Duration
Planning & design	Requirements, UML & ER diagrams	12 days
Backend build	C++ & Java — auth, vendor/inventory, orders	24 days
Routing & jobs	Python — smart routing, reliability scoring	13 days
Frontend build	TypeScript — customer, vendor, delivery, admin portals	21 days
Integration & testing	Real-time notifications, end-to-end testing	12 days
Deployment	Production rollout & documentation	7 days

Figure 8 — Project Gantt Chart

Development timeline split across the languages in use: C++ and Java for the core backend, Python for routing and background jobs, and TypeScript for the frontend portals.

